



Alma Mater Studiorum-Università di Bologna

Scuola di Ingegneria

Fondamenti di Informatica T2

Esercitazione Individuale: CounterFX

Corso di Laurea in Ingegneria Informatica

Anno accademico 2023/2024

Prof. ENRICO DENTI

Dipartimento di Informatica – Scienza e Ingegneria (DISI)



Una vera app grafica

- Nelle scorse lezioni abbiamo visto le basi di JavaFX
 - gestione degli eventi
 - controlli grafici (widget) con relativi mini-esempi
- Tali mini-esempi, però:
 - gestiscono gli eventi in modo diretto (senza controller)
 - essendo molto semplici (per mostrare i singoli widget), non permettono di cogliere la complessità delle vere applicazioni
- Nuovo obiettivo: progettare una vera app grafica per il nostro **Counter** → **applicazione CounterFX**

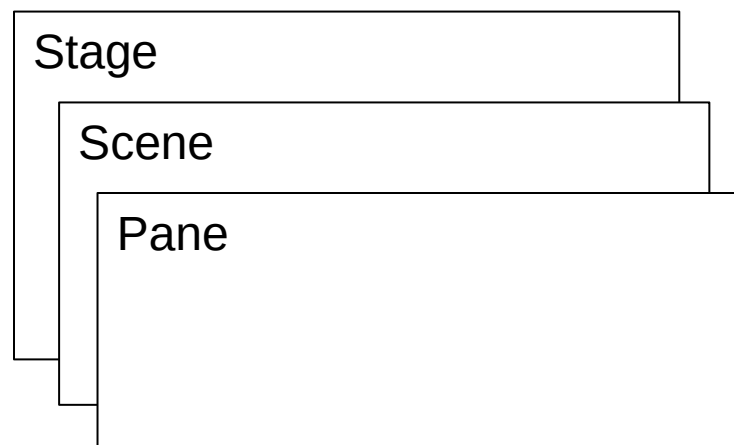
CounterFX: 1ª versione

- Obiettivo: progettare un'interfaccia grafica che gestisca un **Counter** (in realtà, un **Counter2** con anche il decremento) permettendo di:
 - saperne costantemente il valore
 - modificarlo invocando le tre operazioni (reset, inc, dec) con altrettanti appositi pulsanti



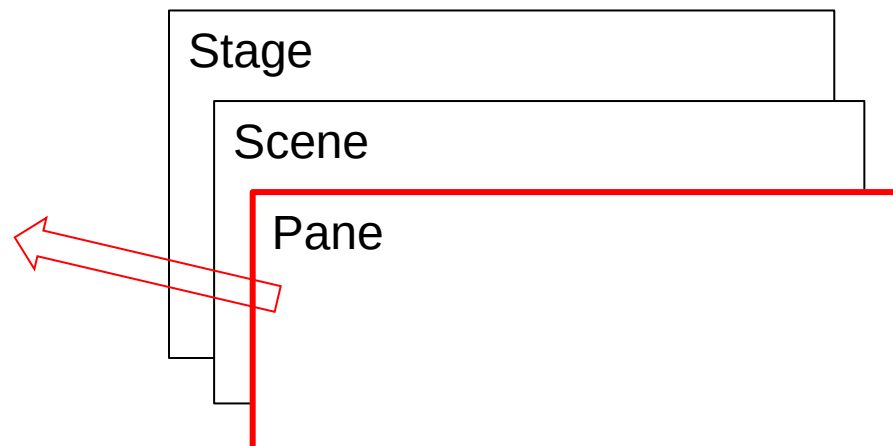
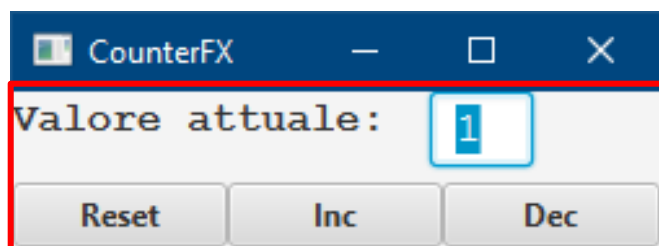
CounterFX: 1ª versione

- Approccio MVC
 - **Model**: il Counter che viene manipolato
 - **View**: un'opportuna app JavaFX con relativi Pane e controlli
 - **Controller**: il «burattinaio» che, sulla base delle richieste della View, manipola il Model
- La view
 - La scena sullo stage è un opportuno pannello (Pane): spesso si si usa/specializza un **BorderPane**
 - Su tale pannello si piazzano i vari widget



CounterFX: 1ª versione

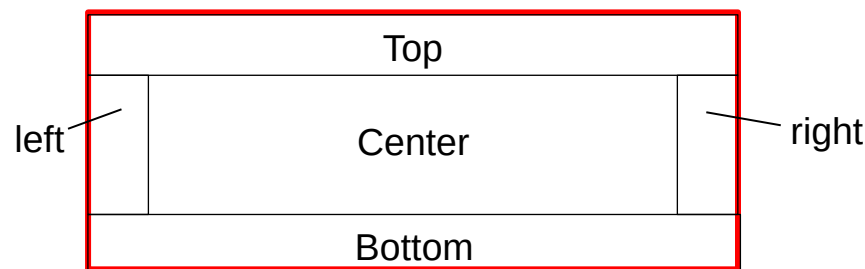
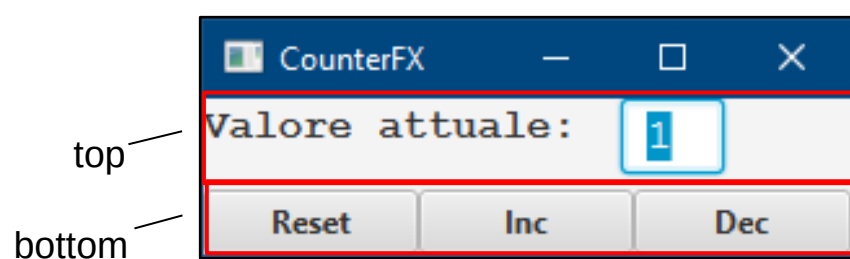
- Il pannello è spesso una specializzazione di **BorderPane**, il più pratico e flessibile dei pannelli



- In questo caso il pannello dovrà contenere:
 - sopra, un'etichetta (**Label**) con, a fianco, un campo di testo (**TextField**) imm modificabile, per mostrare il valore
 - sotto, tre pulsanti (**Button**) per controllare il contatore

CounterFX: 1ª versione

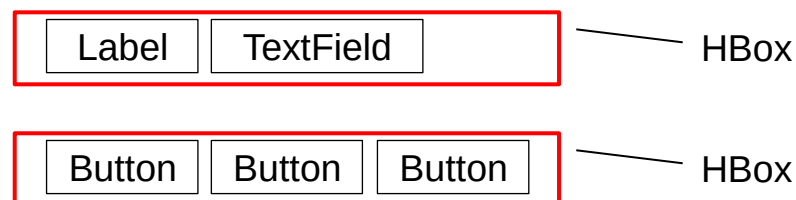
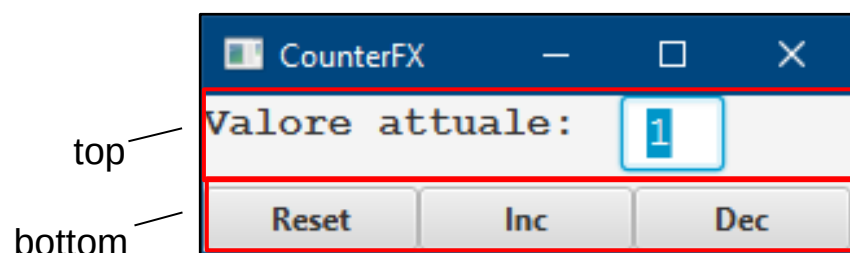
- Nello nostre app, solitamente il pannello è chiamato **MainPane** ed è una sottoclasse custom di BorderPane



- Nell'app in questione, usiamo solo due dei cinque slot:
 - il top per la parte sopra
 - il bottom per la parte sotto

CounterFX: 1ª versione

- Per motivi estetici usiamo inoltre due **HBox**
 - pannelli particolari che dispongono i widget tutti in orizzontale



- Nelle nostre app la parte strutturale è tutta descritta programmaticamente, nel codice
 - non usiamo descrizioni FXML esterne
- Dunque.. forza e coraggio!



CounterFX: 1ª versione

- **HBox** superiore
 - Label, Campo di testo

```
public class MainPane extends BorderPane {  
  
    private Controller controller;  
    private TextField txtValore;  
    private Button reset, inc, dec;  
  
    public MainPane(Controller controller) {  
        this.controller=controller;  
  
        HBox topBox = new HBox();  
        Label titolo = new Label("Valore attuale: ");  
        titolo.setFont(Font.font("Courier New", FontWeight.NORMAL, 16));  
        titolo.setStyle("-fx-font-weight: bold");  
        txtValore = new TextField();  
        txtValore.setPrefWidth(40);  
        txtValore.setFont(Font.font("Courier New", FontWeight.NORMAL, 16));  
        txtValore.setEditable(false);  
        txtValore.setText(controller.getCounterValueAsString());  
        topBox.getChildren().addAll(titolo, new Label(" "), txtValore);  
        this.setTop(topBox);  
    }  
}
```

CounterFX: 1ª versione

- **HBox** inferiore
 - i tre Button (con relativo aggancio del gestore eventi)

```
HBox bottomBox = new HBox();  
// ----- pulsante inc -----  
inc = new Button("Inc");  
inc.setStyle("-fx-font-weight: bold");  
inc.setPrefWidth(100);  
inc.setOnAction(this::incHandle);  
// ----- pulsante dec -----  
dec = new Button("Dec");  
dec.setStyle("-fx-font-weight: bold");  
dec.setPrefWidth(100);  
dec.setOnAction(this::decHandle);  
// ----- pulsante reset -----  
reset = new Button("Reset");  
reset.setStyle("-fx-font-weight: bold");  
reset.setPrefWidth(100);  
reset.setOnAction(this::resetHandle);  
// ----- settaggio pulsanti -----  
bottomBox.getChildren().addAll(reset, inc, dec);  
this.setBottom(bottomBox);  
}
```

In questa versione, **tre gestori eventi distinti** per i vari pulsanti



CounterFX: 1ª versione

- Gestione degli eventi
 - **tre gestori eventi distinti** → non vi è necessità di distinguere la sorgente (*ma vi sarà una certa replicazione di codice...*)
 - **MVC** → i gestori *non agiscono mai direttamente sul model*: si devono sempre rivolgere al **controller**
- Il Controller
 - **incapsula un Counter**, che costituisce lo **stato** dell'applicazione
 - **espone metodi «opportuni»** perché la view possa agire indirettamente sullo stato (ossia, sul Counter incapsulato)
 - incCounter, decCounter, resetCounter, getCounterValueAsString

CounterFX: 1ª versione

- Il controller
 - il costruttore costruisce il Counter inglobato
 - tre metodi delegano le operazioni al Counter interno, senza modifiche
 - `getValue` è leggermente modificato per restituire già il valore come stringa (a beneficio della view)

```
package counter.controller;

import counter.Counter2;

public class Controller {

    private Counter2 counter;

    public Controller() {
        this.counter=new Counter2();
    }

    public void incCounter() {
        this.counter.inc();
    }

    public void decCounter() {
        this.counter.dec();
    }

    public void resetCounter() {
        this.counter.reset();
    }

    public String getCounterValueAsString() {
        return String.valueOf(this.counter.getValue());
    }

}
```

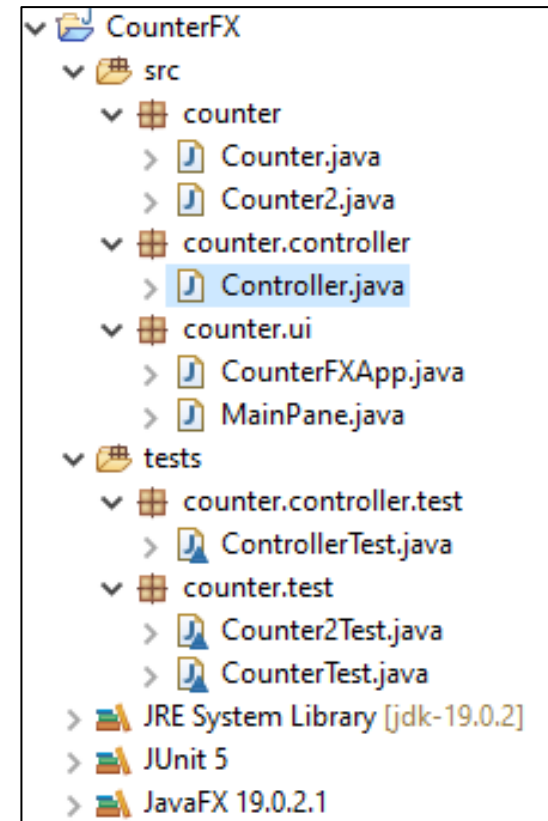
CounterFX: 1ª versione

- Gestione degli eventi
 - tre gestori eventi distinti → non vi è necessità di distinguere la sorgente (*ma vi è una certa replicazione di codice...*)

```
private void incHandle(ActionEvent ev) {  
    controller.incCounter();  
    txtValore.setText(controller.getCounterValueAsString());  
}  
  
private void decHandle(ActionEvent ev) {  
    controller.decCounter();  
    txtValore.setText(controller.getCounterValueAsString());  
}  
  
private void resetHandle(ActionEvent ev) {  
    controller.resetCounter();  
    txtValore.setText(controller.getCounterValueAsString());  
}
```

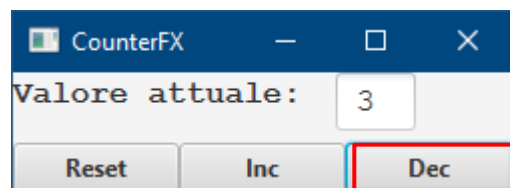
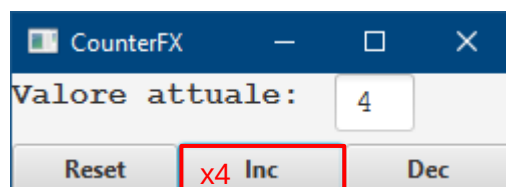
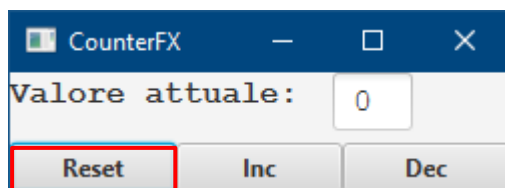
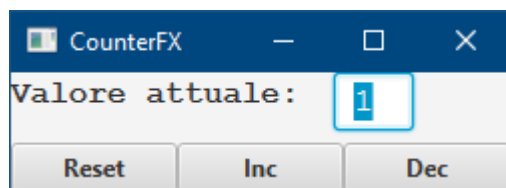
CounterFX: 1ª versione

- Struttura del progetto
 - package distinti per model, controller, e view
 - ulteriori package per i test
 - si collaudano model e controller
 - non la view, perché richiederebbe di simulare l'input dell'utente (esistono framework per fare anche ciò..)



CounterFX: 1ª versione

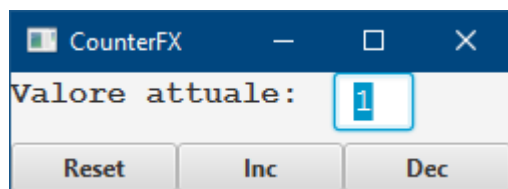
- ..and here it is!



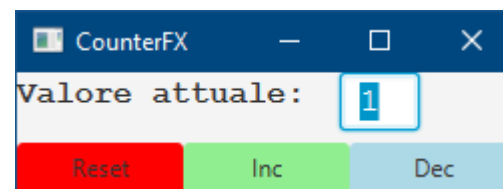
CounterFX: 2ª versione

- Miglioramento: pulsanti colorati

– prima



- nuova versione



- Cosa cambia?
 - solo la view!
 - tutto il resto (e quindi anche i test!) resta intoccato
- Come fare i pulsanti colorati?
 - potremmo applicare gli stili tre volte, ma non è smart
 - IDEA: fare una sottoclasse ad hoc di Button 😊



CounterFX: 2ª versione

- MyButton: il pulsante «a modo nostro»
 - fattorizza le proprietà comuni ai tre pulsanti
 - ma parametrizza il colore

```
public class MyButton extends Button {  
  
    public MyButton(String txt, String color) {  
        super(txt);  
        this.setStyle("-fx-font-weight: bold");  
        this.setPrefWidth(100);  
        this.setStyle("-fx-background-color: " + color);  
    }  
  
}
```

- ciò rende più conciso e leggibile il MainPane
- Con l'occasione, proviamo anche a sperimentare un *gestore eventi unico* anziché tre distinti (sarà meglio..?)

CounterFX: 2ª versione

- Il nuovo MainPane
 - NB: il gestore eventi agganciato è sempre lo stesso

```
HBox bottomBox = new HBox();  
    // ----- pulsante inc -----  
    inc = new MyButton("Inc", "lightgreen");  
    inc.setOnAction(this::myHandle);  
    // ----- pulsante dec -----  
    dec = new MyButton("Dec", "lightblue");  
    dec.setOnAction(this::myHandle);  
    // ----- pulsante resec -----  
    reset = new MyButton("Reset", "red");  
    reset.setOnAction(this::myHandle);  
    // ----- settaggio pulsanti -----  
    bottomBox.getChildren().addAll(reset, inc, dec);  
    this.setBottom(bottomBox);  
}
```

CounterFX: 2ª versione

- Il nuovo gestore eventi
 - stavolta occorre distinguere la sorgente dell'evento
 - si potrebbe usare una serie di *if* (bruttino..)
 - ci divertiamo a sperimentare l'uso di una più gradevole *switch expression* con *pattern matching* 😊

```
@SuppressWarnings("preview")
private void myHandle(ActionEvent ev) {
    switch(ev.getSource()) {
        case Button b when b==inc    -> controller.incCounter();
        case Button b when b==dec    -> controller.decCounter();
        case Button b when b==reset -> controller.resetCounter();
        default -> {}
    }
    txtValore.setText(controller.getCounterValueAsString());
}
```

NB: il caso default
deve essere presente
obbligatoriamente

- Però, gli switch non sono mai una bella soluzione...

CounterFX: 2ª versione

- Non potremmo evitare quello switch?
 - ricorda l'esempio banconote: lo switch è quasi sempre segno che *la rappresentazione dei dati è inadeguata*
 - Per evitarlo occorre *passare al gestore eventi la funzione da chiamare*
 - doppio vantaggio: evitiamo di cablarla nel gestore e in più evitiamo l'inelegante e prolisso switch
 - Le funzioni da chiamare (del controller) hanno tutte la stessa signature
 - consumano non consumano e non restituiscono nulla
- PROBLEMA: non c'è alcuna interfaccia funzionale siffatta ☹



CounterFX: 2ª versione

- NO PROBLEM!

L'interfaccia funzionale mancante la inventiamo noi!

- chiamiamola `Action`: il metodo interno potrà chiamarsi ad esempio `doOp`, non riceve nulla e non restituisce nulla
- *importante ricordare il tag `@FunctionalInterface`*

```
@FunctionalInterface
public interface Action {
    void doOp();
}
```

CounterFX: 2ª versione

- L'aggancio dell'ascoltatore riadattato
 - stavolta la lambda è esplicita, perché dobbiamo passare un argomento di nostra scelta

```
HBox bottomBox = new HBox();  
// ----- pulsante inc -----  
inc = new MyButton("Inc", "lightgreen");  
inc.setOnAction(ev -> myHandle(controller::incCounter));  
// ----- pulsante dec -----  
dec = new MyButton("Dec", "lightblue");  
dec.setOnAction(ev -> myHandle(controller::decCounter));  
// ----- pulsante resec -----  
reset = new MyButton("Reset", "red");  
reset.setOnAction(ev -> myHandle(controller::resetCounter));  
// ----- settaggio pulsanti -----  
bottomBox.getChildren().addAll(reset, inc, dec);  
this.setBottom(bottomBox);
```

CounterFX: 2ª versione

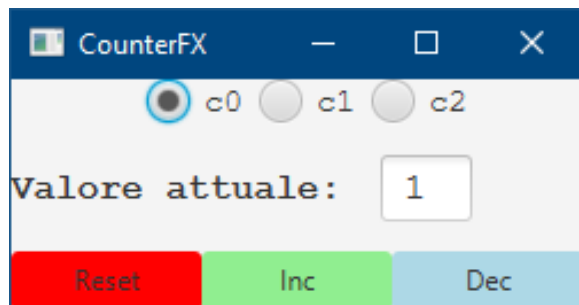
- L'ascoltatore riadattato
 - lo switch scompare e il codice si riduce a UNA riga!! 😊
 - è il sintomo che la via è quella giusta 😊

```
HBox bottomBox = new HBox();  
// ----- pulsante inc -----  
inc = new MyButton("Inc", "lightgreen");  
inc.setOnAction(ev -> myHandle(controller::incCounter));  
// ----- pulsante dec -----  
dec = new MyButton("Dec", "lightblue");  
dec.setOnAction(ev -> myHandle(controller::decCounter));  
// ----- pulsante resec -----  
reset = new MyButton("Reset", "red");  
reset.setOnAction(ev -> myHandle(controller::resetCounter));  
// ----- settaggio pulsanti -----  
bottomBox.getChildren().addAll(reset, inc, dec);  
this.setBottom(bottomBox);
```

```
private void myHandle(Action action) {  
    action.doOp();  
    txtValore.setText(controller.getCounterValueAsString());  
}
```

Ma perché un solo Counter..?

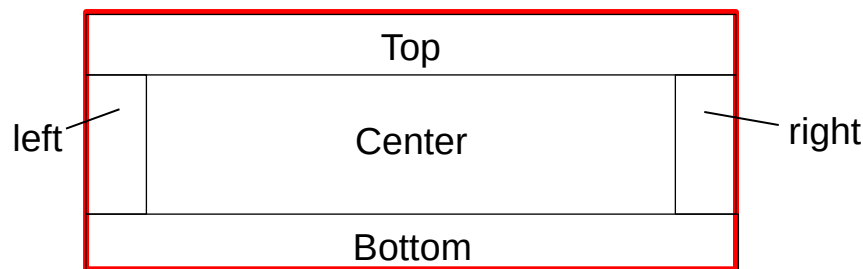
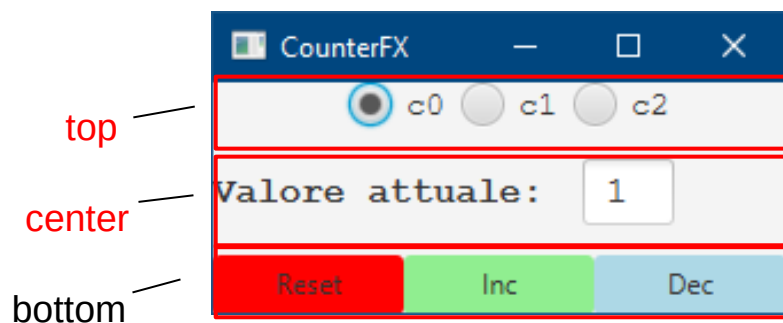
- L'appetito vien mangiando!
 - perché non gestire anche più Counter...?
- Esempio: tre Counter selezionabili con bottoni radio
 - che però «riusino» fondamentalmente la stessa GUI..



- Cosa cambierebbe?

CounterFX: 3ª versione

- Stavolta dobbiamo sfruttare **tre** sezioni del MainPane
 - top, bottom e **center**
 - spostiamo al centro ciò che prima era al top
 - al top, ora, mettiamo i tre RadioButton



- Ipotesi:
 - i pulsanti sotto e il valore mostrato si riferiscono in ogni istante al *contatore correntemente selezionato in alto*

CounterFX: 3ª versione

- Problema: come cambia l'applicazione?
 - il Controller dovrà inglobare *tre* Counter invece di uno
 - quindi, i suoi metodi dovranno prevedere un argomento che specifichi *su quale contatore di debba operare*
- Cosa scegliere come tale argomento?
 - verrebbe da dire «il Counter» su cui operare, *ma sarebbe sbagliato: la view non deve conoscere il model*, non deve neppure sapere di che classi sia fatto!
 - scegliamo perciò di prevedere semplicemente *un intero*, che rappresenti il «numero d'ordine» del Counter su cui operare

CounterFX: 3ª versione

- Il nuovo controller

```
public class Controller {  
  
    private Counter2[] counters;  
    private int index, max;  
  
    public Controller(int max) {  
        this.counters=new Counter2[max];  
        this.max=max;  
        index=0; // the next counter to be created  
    }  
  
    public int getCapacity() {  
        return max;  
    }  
  
    public int newCounter() {  
        if (index<counters.length) { this.counters[index++]=new Counter2(); return index-1;}  
        else throw new IllegalArgumentException("Capacity exceeded");  
    }  
  
    public void incCounter(int i) {  
        if (i>=0 && i<counters.length) this.counters[i].inc();  
    }  
  
    public void decCounter(int i) {  
        if (i>=0 && i<counters.length) this.counters[i].dec();  
    }  
  
    public void resetCounter(int i) {  
        if (i>=0 && i<counters.length) this.counters[i].reset();  
    }  
  
    public String getCounterValueAsString(int i) {  
        return (i>=0 && i<counters.length) ? String.valueOf(this.counters[i].getValue()) : "N/A";  
    }  
}
```

Prevede internamente
un array di Counter

I metodi ricevono l'indice
(intero) del contatore su
cui operare: così, la view
non conosce il model

CounterFX: 3ª versione

- Alla luce di ciò, quali dati ci servono nel MainPane?
 - un array o lista di interi (che rappresenta «i contatori»)
 - un indice del contatore correntemente selezionato
 - i tre RadioButton con relativo ToggleGroup
- Cambia qualcosa nel comportamento?
 - l'ascoltatore che c'è già (quello dei tre pulsanti Reset/Inc/Dec) è concettualmente identico, ma (nella versione senza switch) *cambia leggermente la signature* perché le funzioni del controller ora consumano tutte un intero
 - **occorre inoltre un nuovo ascoltatore per i tre Radiobutton**, per aggiornare l'indice del «contatore corrente»

CounterFX: 3ª versione

- Anche qui possiamo decidere di eliminare lo switch
- Per evitarlo occorre *passare al gestore eventi la funzione da chiamare*
 - doppio vantaggio: evitiamo di cablarla nel gestore e in più evitiamo l'inelegante e prolisso switch
- Le funzioni da chiamare (del controller) hanno tutte la stessa signature
 - stavolta consumano un intero e non restituiscono nulla
→ un chiaro caso di **IntConsumer** 😊

CounterFX: 3ª versione

- L'aggancio dell'ascoltatore non cambia
 - la lambda resta esplicita, perché dobbiamo passare un argomento di nostra scelta

```
HBox bottomBox = new HBox();
// ----- pulsante inc -----
inc = new MyButton("Inc", "lightgreen");
inc.setOnAction(ev -> myHandle(controller::incCounter));
// ----- pulsante dec -----
dec = new MyButton("Dec", "lightblue");
dec.setOnAction(ev -> myHandle(controller::decCounter));
// ----- pulsante resec -----
reset = new MyButton("Reset", "red");
reset.setOnAction(ev -> myHandle(controller::resetCounter));
// ----- settaggio pulsanti -----
bottomBox.getChildren().addAll(reset, inc, dec);
this.setBottom(bottomBox);
```

CounterFX: 3ª versione

- L'ascoltatore riadattato
 - stavolta l'azione è di tipo `IntConsumer`, quindi il metodo interno da invocare si chiama `accept`

```
HBox bottomBox = new HBox();
// ----- pulsante inc -----
inc = new MyButton("Inc", "lightgreen");
inc.setOnAction(ev -> myHandle(controller::incCounter));
// ----- pulsante dec -----
dec = new MyButton("Dec", "lightblue");
dec.setOnAction(ev -> myHandle(controller::decCounter));
// ----- pulsante resec -----
reset = new MyButton("Reset", "red");
reset.setOnAction(ev -> myHandle(controller::resetCounter));
// ----- settaggio pulsanti -----
bottomBox.getChildren().addAll(reset, inc, dec);
this.setBottom(bottomBox);
```

```
private void myHandle(IntConsumer action) {
    action.accept(currentCounter);
    txtValore.setText(controller.getCounterValueAsString(currentCounter));
}
```

CounterFX: 3ª versione

- La nuova sezione «top» del MainPane
 - cerchiamo di evitare codice ripetuto, usiamo cicli e metodi di utilità per fattorizzare il codice comune

```
private RadioButton[] buttonsCounters;  
private int[] counters;  
private int currentCounter;  
  
public MainPane(Controller controller) {  
    this.controller=controller;  
    buttonsCounters = new RadioButton[controller.getCapacity()];  
    counters = new int[controller.getCapacity()];  
    //  
    HBox topBox = new HBox();  
    tg = new ToggleGroup();  
    topBox.setAlignment(Pos.CENTER);  
    for(int i=0; i<3; i++) {  
        buttonsCounters[i] = new RadioButton("c" + i);  
        buttonsCounters[i].setFont(Font.font("Courier New", FontWeight.NORMAL, 14));  
        buttonsCounters[i].setToggleGroup(tg);  
        counters[i] = controller.newCounter();  
        topBox.getChildren().addAll(new Label(" "), buttonsCounters[i]);  
        buttonsCounters[i].setOnAction(this::setCurrentCounter);  
    }  
    currentCounter = 0;  
    tg.selectToggle(buttonsCounters[currentCounter]);  
    this.setTop(topBox);  
}
```

Prevede un array di
«indici di contatori»

Creazione e configu-
razione RadioButton

Nuovo ulteriore listener

CounterFX: 3ª versione

- Il nuovo ascoltatore
 - ascolta i RadioButton e aggiorna l'indice del «counter corrente»
 - MA... anche qui uno switch...?

```
@SuppressWarnings("preview")
private void setCurrentCounter(ActionEvent ev) {
    currentCounter = switch(ev.getSource()) {
        case RadioButton b when b==buttonsCounters[0] -> 0;
        case RadioButton b when b==buttonsCounters[1] -> 1;
        case RadioButton b when b==buttonsCounters[2] -> 2;
        default -> 0;
    };
    txtValore.setText(controller.getCounterValueAsString(currentCounter));
}
```

CounterFX: 3ª versione

- E se non volessimo quell'orrido switch?
 - bisognerebbe che il RadioButton «sapesse già» il suo valore, così da non dover distinguere caso per caso per calcolarlo
- Come fare?
 - prima (pessima) idea: sfruttare la label del RadioButton stesso
 - ovvero, trafficare sulla stringa «c0», «c1», etc. per ricavare il valore associato al singolo RadioButton
 - pessimo, perché se la label cambia anche di pochissimo, crolla tutto!
 - idea migliore: farci il nostro RadioButton evoluto, che incapsuli già il corrispondente valore intero
 - come nel caro vecchio caso delle banconote...! ☺

CounterFX: 3ª versione

- MyRadioButton
 - estende RadioButton incapsulando il valore intero
- Il MainPane usa tale classe

```
public class MyRadioButton extends RadioButton {  
  
    private int value;  
  
    public MyRadioButton(String txt, int value) {  
        super(txt);  
        this.value=value;  
    }  
  
    public int getValue() {  
        return value;  
    }  
  
}
```

```
HBox topBox = new HBox();  
tg = new ToggleGroup();  
topBox.setAlignment(Pos.CENTER);  
for(int i=0; i<3; i++) {  
    buttonsCounters[i] = new MyRadioButton("c" + i, i);  
    buttonsCounters[i].setFont(Font.font("Courier New", FontWeight.NORMAL, 14));  
    buttonsCounters[i].setToggleGroup(tg);  
    counters[i] = controller.newCounter();  
    topBox.getChildren().addAll(new Label(" "), buttonsCounters[i]);  
    buttonsCounters[i].setOnAction(this::setCurrentCounter);  
}  
currentCounter = 0;  
tg.selectToggle(buttonsCounters[currentCounter]);  
this.setTop(topBox);
```

CounterFX: 3ª versione

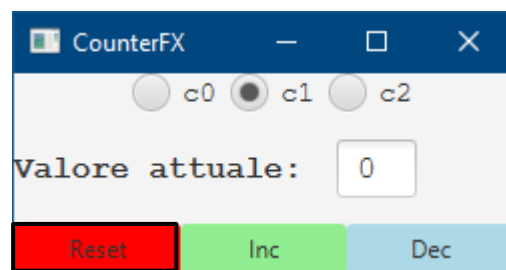
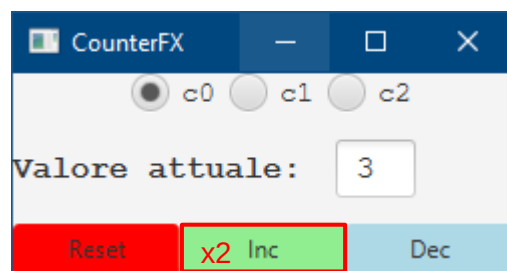
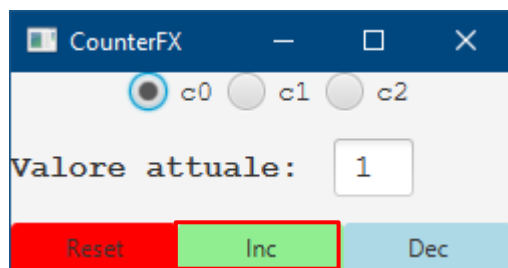
- Il nuovo ascoltatore riadattato
 - anziché getSource, sperimentiamo getSelectedToggle per recuperare il pulsante che ha scatenato l'evento
 - il risultato però è un Toggle (superclasse di RadioButton) → va convertito in MyRadioButton (in modo sicuro!)
 - a quel punto, anche qui il codice si riduce a UNA riga! 😊

```
private void setCurrentCounter(ActionEvent ev) {  
    if (tg.getSelectedToggle() instanceof MyRadioButton b) {  
        currentCounter = b.getValue();  
    }  
    txtValore.setText(controller.getCounterValueAsString(currentCounter));  
}
```

- Morale: lo switch non è quasi mai *davvero* necessario!

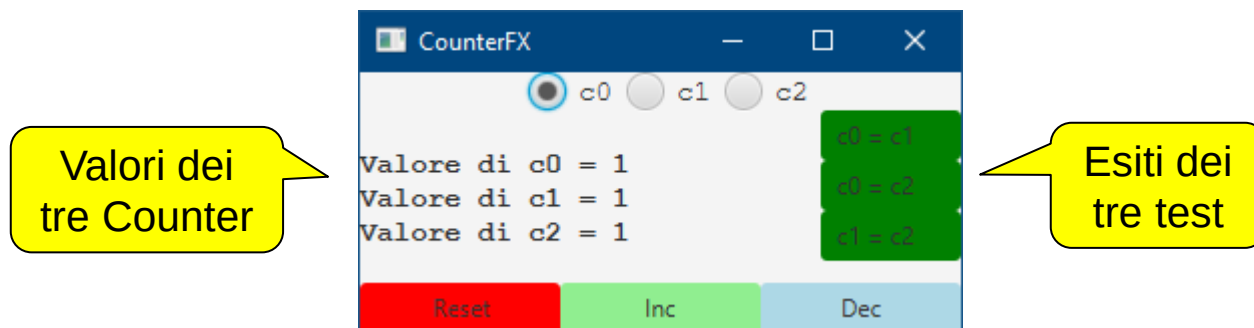
CounterFX: 3ª versione

- ..and the winner is...



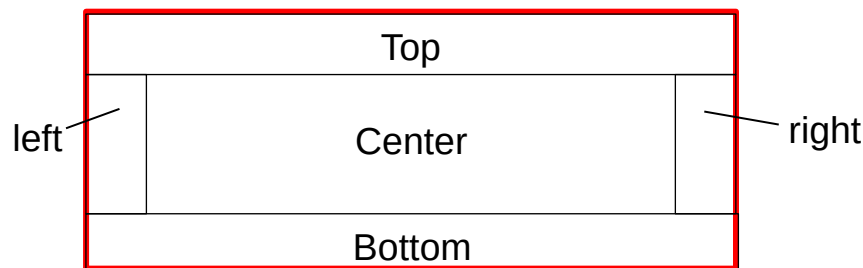
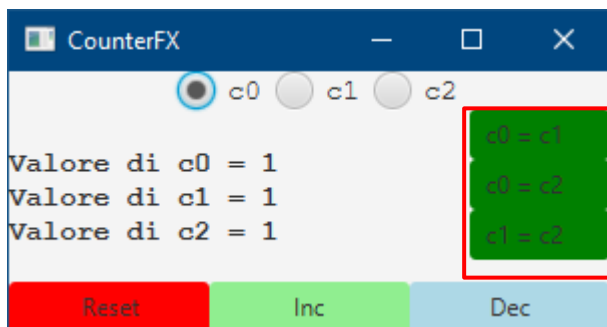
Ma perché fermarsi qui..? 😊

- Ci sarebbero anche altri metodi, come **equals...** 😊
 - non li abbiamo ancora inclusi nella view!
- Ma per farlo occorre **manipolare contemporaneamente più Counter**
 - questo implica rivedere la GUI un po' più in profondità
 - si può ancora *modificare* un solo Counter per volta, MA **non basta più mostrare su un solo Counter per volta!**



CounterFX: 4ª versione

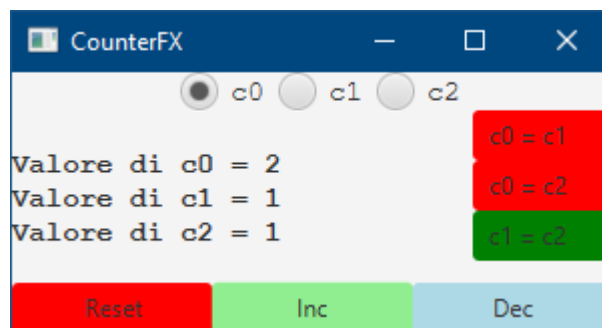
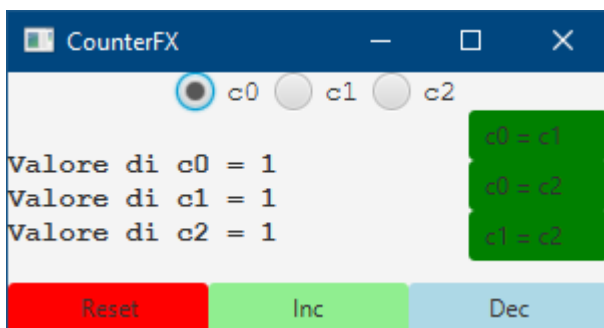
- Stavolta si sfruttano **quattro** sezioni del MainPane
 - top, bottom, center e **right**



- Novità:
 - inseriamo sulla destra tre TextField che mostrino l'esito dei tre test sui tre Counter (c0=c1, c0=c2, c1=c2) tramite apposito colore di sfondo verde/rosso

CounterFX: 4ª versione

- Cosa cambia?
 - NON il controller, perché gestiva già più Counter
 - **SOLO la view**, che invece ne mostrava uno solo
- Come mostriamo il risultato di `equals`?
 - inseriamo tre TextField coi tre test e ne controlliamo il colore di sfondo (verde/rosso) in base all'esito di `equals`



Esiti dei
tre test



CounterFX: 4ª versione

- Cosa cambia nella view?
 - ci vogliono tre label per il valore dei Counter (anziché una)
NB con l'occasione usiamo solo Label anziché Label+TextField
 - parametrizziamo tutto il parametrizzabile e fattorizziamo tutto il fattorizzabile, così da avere codice pulito e leggibile
- In questa versione, l'ottimizzazione dei listener eliminando gli switch è lasciata per esercizio al lettore
 - introduciamo però una serie di metodi di utilità
 - **due factory methods** per incapsulare la costruzione dei bottoni (`makeMyButton`) e dei TextField (`makeEqField`)
 - **due utility methods** per incapsulare l'aggiornamento di label (`updateValueLabels`) e TextField (`updateEqFields`)

CounterFX: 4ª versione

```
public class MainPane extends BorderPane {

    private Controller controller;
    private TextField txtEq12, txtEq13, txtEq23;
    private Label labelValore1, labelValore2, labelValore3;
    private Button reset, inc, dec;
    private ToggleGroup tg;
    private RadioButton[] buttonsCounters;
    private int[] counters;
    private int currentCounter;

    public MainPane(Controller controller) {
        this.controller=controller;
        buttonsCounters = new RadioButton[controller.getCapacity()];
        counters = new int[controller.getCapacity()];

        HBox topBox = new HBox();
        tg = new ToggleGroup();
        topBox.setAlignment(Pos.CENTER);
        for(int i=0; i<3; i++) {
            buttonsCounters[i] = new RadioButton("c" + i);
            buttonsCounters[i].setFont(Font.font("Courier New", FontWeight.NORMAL, 14));
            buttonsCounters[i].setToggleGroup(tg);
            counters[i] = controller.newCounter();
            topBox.getChildren().addAll(new Label(" "), buttonsCounters[i]);
            buttonsCounters[i].setOnAction(this::setCurrentCounter);
        }
        currentCounter = 0;
        tg.selectToggle(buttonsCounters[currentCounter]);
        this.setTop(topBox);
    }
}
```

Si potrebbero usare i
MyRadioButton

CounterFX: 4^a versione

```
VBox centerBox = new VBox();
centerBox.setAlignment(Pos.CENTER_LEFT);
labelValore1 = new Label();
labelValore2 = new Label();
labelValore3 = new Label();
updateValueLabels();
centerBox.getChildren().addAll(labelValore1, labelValore2, labelValore3);
this.setCenter(centerBox);

HBox bottomBox = new HBox();
// ----- pulsanti -----
inc = makeMyButton("Inc", "lightgreen", this::buttonHandle);
dec = makeMyButton("Dec", "lightblue", this::buttonHandle);
reset = makeMyButton("Reset", "red", this::buttonHandle);
// ----- settaggio pulsanti -----
bottomBox.getChildren().addAll(reset, inc, dec);
this.setBottom(bottomBox);

VBox rightBox = new VBox();
txtEq12 = makeEqField("c0 = c1");
txtEq13 = makeEqField("c0 = c2");
txtEq23 = makeEqField("c1 = c2");
rightBox.getChildren().addAll(txtEq12, txtEq13, txtEq23);
this.setRight(rightBox);
updateEqFields();
}
```

Da aggiustare se si vuole
eliminare lo switch come
nella 2^a versione

CounterFX: 4ª versione

- Il nuovo ascoltatore
 - notare le fattorizzazioni tramite metodi di utilità

```
@SuppressWarnings("preview")
private void buttonHandle(ActionEvent ev) {
    switch(ev.getSource()) {
        case Button b when b==inc    -> controller.incCounter(currentCounter);
        case Button b when b==dec    -> controller.decCounter(currentCounter);
        case Button b when b==reset  -> controller.resetCounter(currentCounter);
        default -> {}
    }
    updateValueLabels();
    updateEqFields();
}

@SuppressWarnings("preview")
private void setCurrentCounter(ActionEvent ev) {
    currentCounter = switch(ev.getSource()) {
        case RadioButton b when b==buttonsCounters[0] -> 0;
        case RadioButton b when b==buttonsCounters[1] -> 1;
        case RadioButton b when b==buttonsCounters[2] -> 2;
        default -> 0;
    };
}
```

Switch eliminabile come
nella 2ª versione

Switch eliminabile
usando MyRadioButton



CounterFX: 4^a versione

- Utility methods
 - i due factory methods

```
private MyButton makeMyButton(String text, String backgroundColor, EventHandler<ActionEvent> handler) {  
    MyButton inc = new MyButton(text, backgroundColor);  
    inc.setOnAction(handler);  
    return inc;  
}  
  
private TextField makeEqField(String text) {  
    TextField txtField = new TextField();  
    txtField.setStyle("-fx-font-weight: bold");  
    txtField.setPrefWidth(70);  
    txtField.setEditable(false);  
    txtField.setText(text);  
    return txtField;  
}
```

CounterFX: 4ª versione

- Utility methods
 - i due update methods (e relativi helper)

```
private void updateEqFields() {
    updateEqField(txtEq12, 0,1);
    updateEqField(txtEq13, 0,2);
    updateEqField(txtEq23, 1,2);
}

private void updateEqField(TextField txt, int idx1, int idx2) {
    txt.setStyle("-fx-background-color: " + (
        controller.getCounterValueAsString(idx1).equals(controller.getCounterValueAsString(idx2)) ? "green" : "red"));
}

private void updateValueLabels() {
    updateValueLabel(labelValore1, "Valore di c0 = ", controller.getCounterValueAsString(0));
    updateValueLabel(labelValore2, "Valore di c1 = ", controller.getCounterValueAsString(1));
    updateValueLabel(labelValore3, "Valore di c2 = ", controller.getCounterValueAsString(2));
}

private void updateValueLabel(Label label, String prefix, String value) {
    label.setPrefWidth(160);
    label.setFont(Font.font("Courier New", FontWeight.BOLD, 14));
    label.setText(prefix + value);
}
```

Si potrebbe fattorizzare ulteriormente...? 😊

CounterFX: 4^a versione

- Il gran finale!

